

Construction of NFA for a r.e. using Thompson's construction method

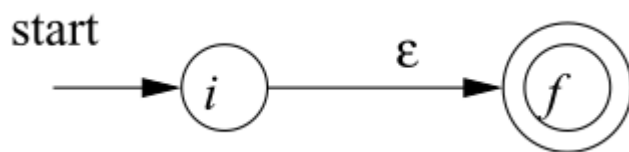
Input: A r.e. r over an alphabet Σ .

Output: An NFA $N(r)$ corresponding to r.e. r .

Method:

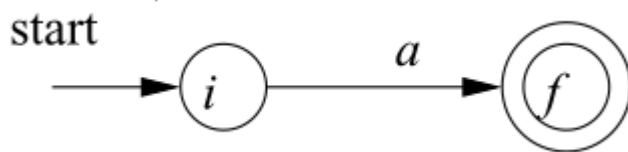
We first parse r into its constituent sub-expressions, then using rules 1 and 2 below, we construct NFAs for each of the basic symbols in r . Then guided by the syntactic structure of r.e. r , we construct these NFAs inductively using rule 3. The rules are:

1. For ϵ , construct the NFA.

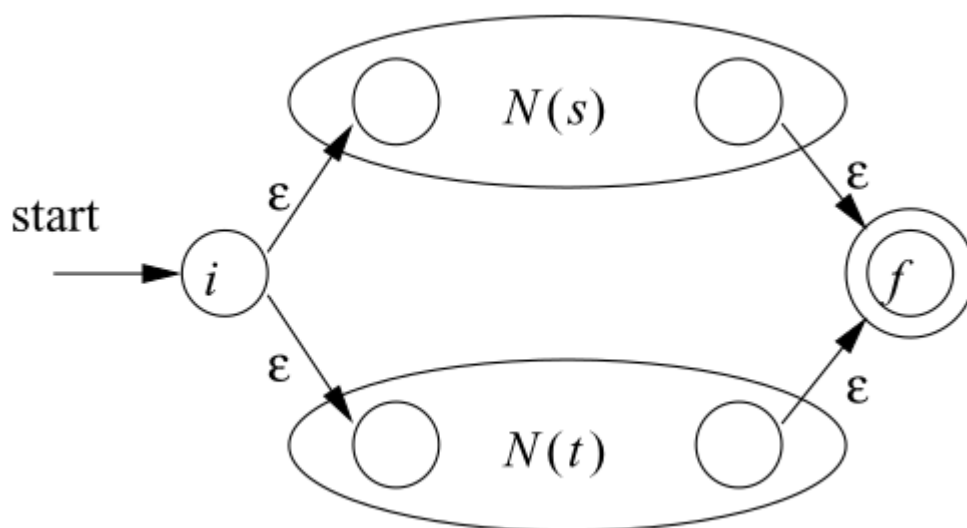


2. Here, i is a new start state and f is a new accepting state. Clearly, this NFA recognizes $\{\epsilon\}$.

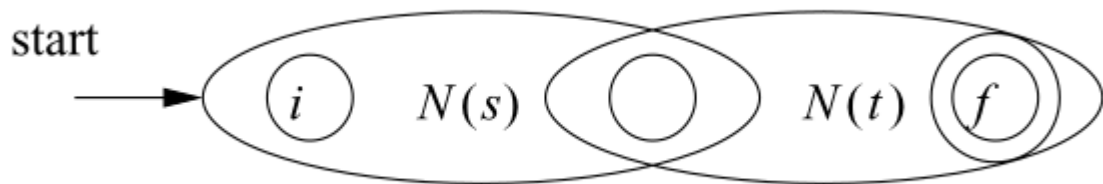
3. For a in Σ , construct the NFA. Again, i is a new start state and f is a new accepting state. This machine recognizes $\{a\}$.



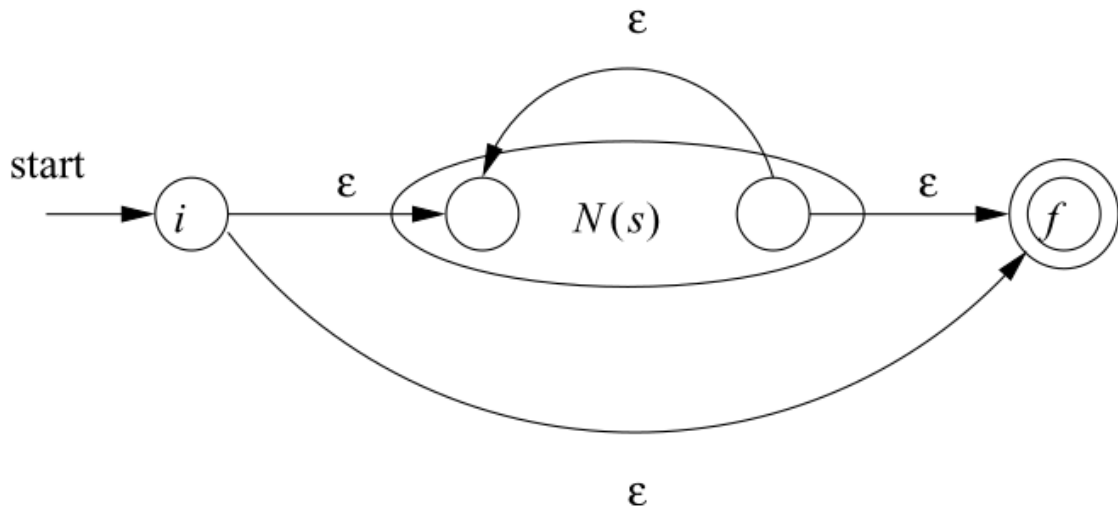
4. Suppose $N(s)$ and $N(t)$ are NFAs for regular expressions s and t . For the regular expression $s|t$, construct the following composite NFA $N(s|t)$.



5. For the regular expression st , construct the NFA $N(st)$.



6. For the r.e. s^* , construct the NFA $N(s^*)$.



7. For the parenthesized r.e., use $N(s)$ itself as the NFA.

Simulating an NFA

Input: An NFA N , constructed by Thompson's algorithm and an input string x which is terminated by an EOF character. N has the start state S_0 and a set of accepting states F .

state

Output: The answer "yes" if N accepts x else "no".

Method:

$s = \epsilon\text{-closure}(\{S_0\});$

$a = \text{getchar}();$

while $a \neq \text{EOF}$ do

begin

$s = \epsilon\text{-closure}(\text{move}(s, a))$

$a = \text{getchar}();$

end

if $s \cap F \neq \emptyset$

then return "yes"

else return "no"

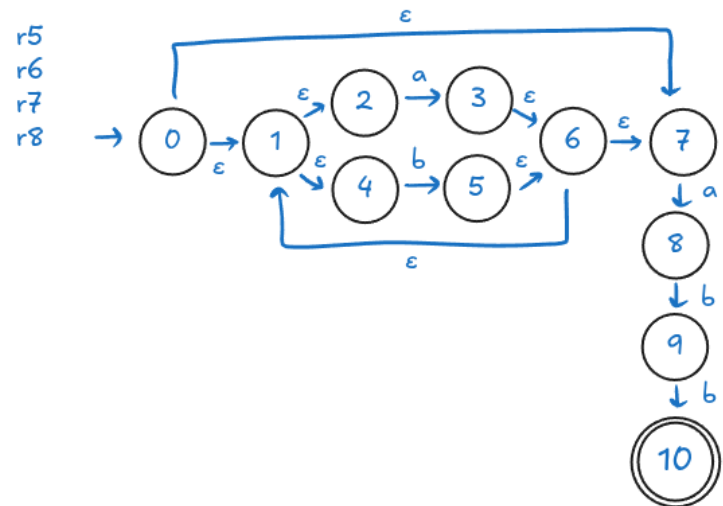
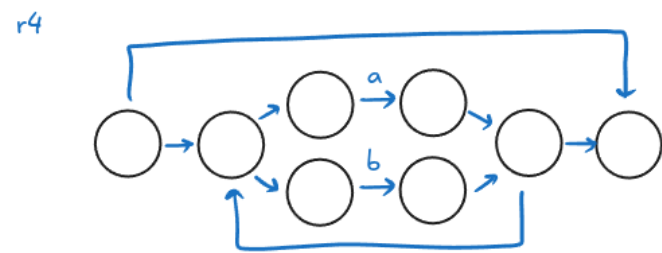
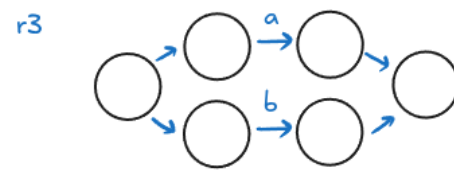
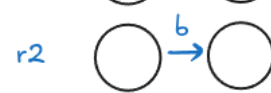
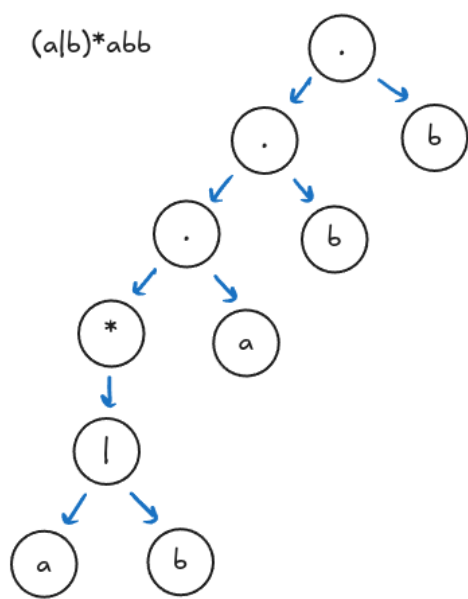


Figure 1: NFA corresponding $(a/b)^*abb$

$(a/b)^*ab(a/b)^*$

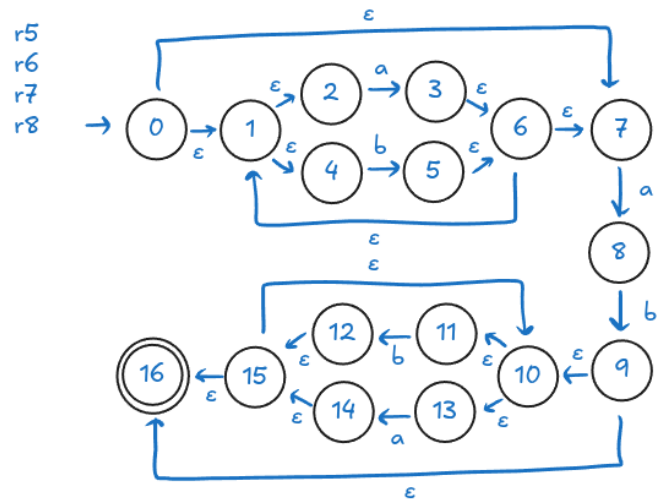
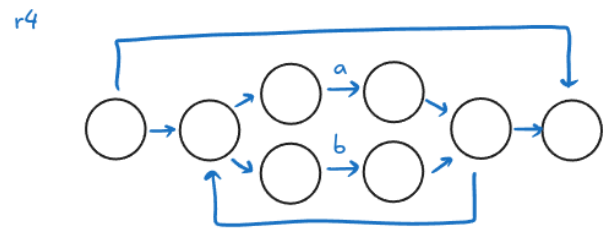
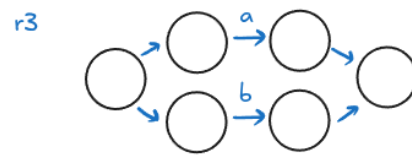
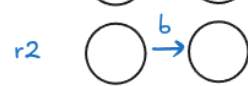
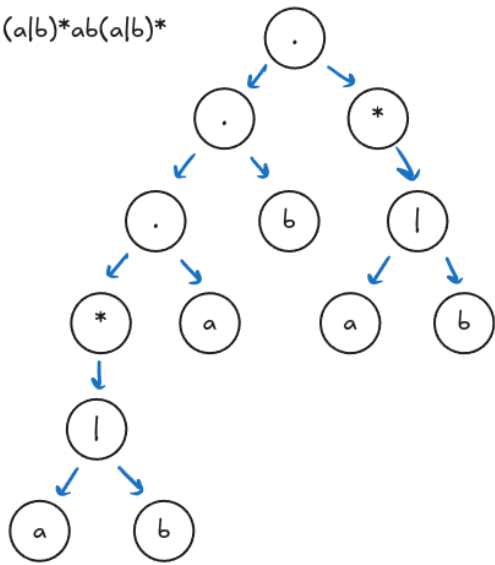


Figure 2: NFA corresponding $(a/b)^*ab(a/b)^*$